INTELIGENCIA ARTIFICIAL · ENTREGA FINAL

Reconocimiento de dígitos mediante búsqueda heurística y algoritmos genéticos

Clasificación de una imagen de 28 × 28 píxeles recorriendo un grafo de 12 nodos (UNKNOWN, dígitos 0–9 y NULL) con BFS, DFS y A*, y ajuste de los pesos de los rasgos con un algoritmo genético.

Contenido

Sección 1. Planteamiento del problema de búsqueda (a–f)

Sección 2. Grafo representativo (12 nodos, pesos 1–10 y justificación)

Sección 3. Algoritmos: BFS/DFS, A*, algoritmo genético y flujo completo

Sección 4. Programa funcional en HTML (animación, 25 pruebas, optimización)

Resultados medidos, limitaciones y publicación en GitHub Pages

24 / 25 aciertos en las 25 pruebas	**96 %** en 500 imágenes de prueba nuevas	**80 % → 96 %** pesos manuales → pesos del AG	**A*: 2 de 11** nodos verificados (ej. «8»)

Resultados medidos con el programa entregado (imágenes sintéticas generadas con fuentes del navegador; ver limitaciones en la Sección 4).

SECCIÓN 1. Planteamiento del problema de búsqueda

Problema. Dada una imagen de 28 × 28 píxeles que contiene un dígito, decidir cuál de los dígitos 0–9 representa, o devolver **NULL** si ninguno se parece lo suficiente. El problema se modela como una búsqueda en un grafo: se parte de un estado desconocido (UNKNOWN), se evalúan los candidatos y se elige el de menor coste de comparación de rasgos.

a) Estado inicial

El estado inicial es la **imagen de entrada** aún sin interpretar: un archivo PNG o JPG que se reduce a 28 × 28 píxeles. Antes de cualquier comparación se normaliza: (1) se pasa a escala de grises; (2) se estima el color de fondo con el borde de la imagen y se separa el trazo (si el trazo es claro sobre fondo oscuro también funciona); (3) se eliminan motas de ruido; (4) se recorta el trazo, se escala para que quepa en un cuadro de 20 × 20 y se centra en 28 × 28.

$$S_0 = (\ I,\ \text{rasgos sin calcular},\ \text{nodo} = \text{UNKNOWN}\ )$$

Ejemplo: si se carga la imagen de un 8, el sistema no asume que es un 8; primero calcula sus rasgos y después los compara con los diez candidatos.

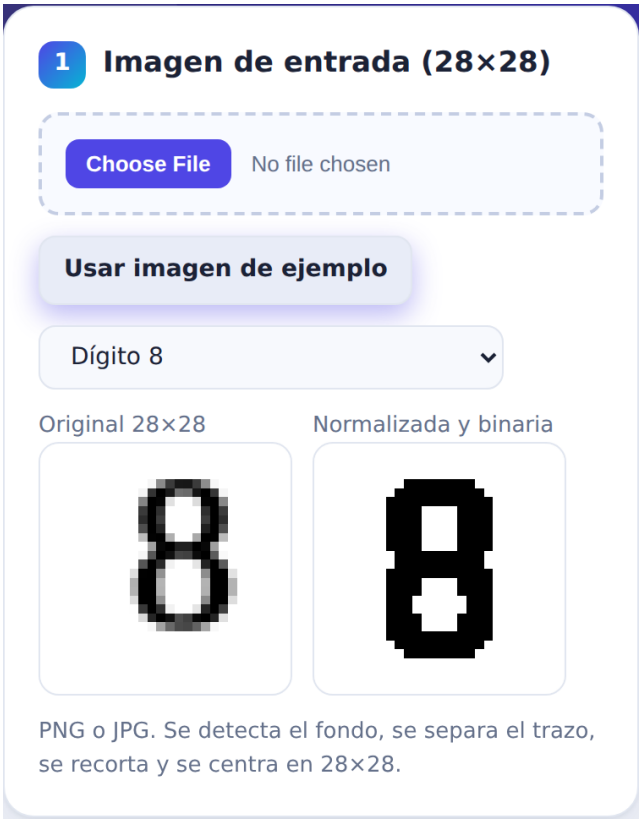


Figura 1. Imagen de entrada (28 × 28) y su versión binaria normalizada, tal como las muestra el programa.

b) Estados (interpretaciones parciales durante la extracción de rasgos)

Cada etapa de la extracción produce una interpretación parcial de la imagen. El estado de búsqueda posterior (candidatos) se construye sobre el vector completo de rasgos.

Estado	Interpretación parcial	Información disponible
E0 · Imagen cruda	Píxeles RGB 28 × 28	Sin separar trazo y fondo
E1 · Binaria	Trazo = 1, fondo = 0	Umbral respecto al fondo del borde
E2 · Normalizada	Trazo recortado, escalado y centrado	Invariante a posición y tamaño
E3 · Huecos	Zonas de fondo encerradas por el trazo	H = número de huecos
E4 · Simetría	Parecido entre mitades izquierda y derecha	S ∈ [0, 1]
E5 · Densidad regional	Proporción de trazo en cada una de 16 regiones (rejilla 4 × 4)	$D_1 \ldots D_{16}$, y densidad total
E6 · Extremos	Puntos terminales del esqueleto del trazo	E = número de endpoints
E7 · Componentes	Partes conexas independientes del trazo	C = número de componentes
E8 · Vector de rasgos	Descripción numérica completa de la imagen	(H, S, D, E, C) listo para comparar
UNKNOWN → 0…9, NULL	Estados de búsqueda: candidato generado, candidato verificado, objetivo	Coste g, estimación h, coste real J

c) Acciones (operaciones de extracción de rasgos y de búsqueda)

Acción	Qué hace
Cargar y reducir	Lee el archivo y lo ajusta a 28 × 28 sin deformar su proporción.
Binarizar y limpiar	Separa trazo y fondo; elimina puntos aislados menores de 4 píxeles y poros de fondo menores de 4 píxeles.
Normalizar	Recorta el trazo, lo escala a 20 px y lo centra.
Contar huecos	Etiqueta el fondo (4-conexo) y cuenta las regiones que no tocan el borde.
Medir simetría	Compara cada píxel de trazo con su espejo horizontal: S = coincidencias / píxeles de trazo.
Medir densidad regional	Divide en 16 regiones de 7 × 7 y calcula la proporción de trazo en cada una.
Detectar extremos	Adelgaza el trazo (algoritmo de Zhang–Suen) y cuenta los píxeles del esqueleto con un solo vecino.
Contar componentes	Etiqueta el trazo (8-conexo) e ignora componentes de menos de 3 píxeles.
Expandir UNKNOWN	Genera los 11 sucesores: los diez dígitos y NULL.
Calcular g, h y J	g y h con rasgos parciales; J con el coste completo del candidato.
Verificar candidato	Completa el coste del candidato (añade componentes) y lo convierte en objetivo.
Seleccionar y validar	Elige el menor coste y comprueba J ≤ τ; si no, devuelve NULL.

d) Pruebas de objetivo (condición para decidir un dígito)

Sea I la imagen de entrada y J(I, d) el coste de compararla con el dígito d. El sistema acepta un dígito cuando es el de menor coste y ese coste no supera el umbral τ:

$$d^* = \operatorname{argmin}_{d \in \{0,\ldots,9\}} J(I, d) \cdot \text{se devuelve } d^* \text{ si } J(I, d^*) \leq \tau, \text{ y NULL en caso contrario}$$

En el grafo, NULL es un nodo objetivo más, con coste fijo τ. Así, la prueba de objetivo de A* («el nodo extraído de la frontera es un objetivo») ya incluye la decisión de rechazo: si todos los dígitos cuestan más que τ, el primer objetivo extraído es NULL. El umbral τ = 0,30 es una decisión de diseño calibrada con pruebas (ver Sección 4); no es una probabilidad.

e) Función de coste (medida de calidad del *match*)

Compara los rasgos de la imagen (I) con los de un prototipo del dígito (d). Cuanto menor es el coste, mejor es la coincidencia:

$$J(I,d) = w_H \cdot |\tilde{H}_I - \tilde{H}_d| + w_S \cdot |S_I - S_d| + w_D \cdot (1/16) \cdot \Sigma_{r=1..16} |D_{I,r} - D_{d,r}| + w_E \cdot |\tilde{E}_I - \tilde{E}_d| + w_C \cdot |\tilde{C}_I - \tilde{C}_d| + 0{,}25 \cdot |D^{tot}_I - D^{tot}_d|$$

Los rasgos de conteo se normalizan a [0, 1]: $\tilde{H} = \text{mín}(H, 3)/3$, $\tilde{E} = \text{mín}(E, 6)/6$, $\tilde{C} = \text{mín}(C, 4)/4$. Cada dígito tiene diez variantes de prototipo (cinco tipografías, normal y negrita) y el coste del dígito es el de su variante más parecida: $J(I,d) = \text{mín}_v\, J(I, d_v)$. Los pesos w son el cromosoma del algoritmo genético; los valores finales son **w = [0,13; 0,56; 4,21; 0,12; 2,39]** (w_H, w_S, w_D, w_E, w_C).

f) Justificación: ¿búsqueda ciega o heurística?

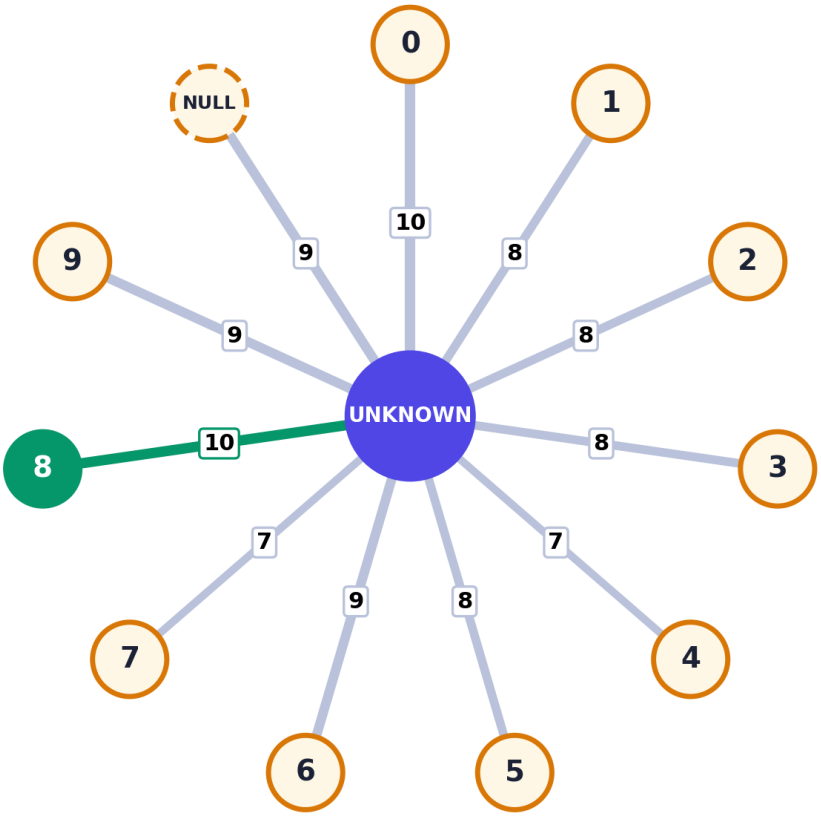
Es un problema de búsqueda porque hay un estado inicial (UNKNOWN), un conjunto de sucesores (los candidatos), acciones que los evalúan, un coste y una prueba de objetivo. Admite dos enfoques:

- **Búsqueda ciega (BFS, DFS).** Recorren los candidatos en un orden fijo (cola o pila) sin usar los rasgos de la imagen para decidir cuál mirar primero; en este grafo acaban verificando los 11 sucesores.
- **Búsqueda heurística (A*).** Antes de verificar un candidato ya se dispone de una estimación barata, h(n), basada en huecos, simetría y extremos. Con f = g + h se verifican primero los candidatos prometedores y se descarta el resto sin completar su coste. Es la opción elegida y la que se compara con BFS/DFS.

Precisión: BFS y DFS no minimizan costes en grafos ponderados; por eso, tras recorrer los candidatos, el programa elige explícitamente el de menor J. A* sí es óptimo porque h es admisible (Sección 3.2).

SECCIÓN 2. Grafo representativo

Estructura. El grafo tiene **12 nodos**: el nodo de inicio **UNKNOWN**, los diez nodos objetivo **0, 1, …, 9** y el nodo objetivo **NULL** (rechazo). Hay 11 aristas ponderadas que parten de UNKNOWN. El peso de cada arista es la **similitud de rasgos** entre la imagen y el dígito, en la escala 1–10 (10 = muy parecido); para los algoritmos de coste mínimo se usa el coste c = 11 − s.



Nodo inicial: UNKNOWN · Nodos objetivo: dígitos 0–9 y NULL · Peso = similitud de rasgos (1–10)
Ejemplo con la imagen de un «8»; camino final en verde: UNKNOWN → 8

Figura 2. Grafo representativo con los pesos reales calculados por el programa para la imagen de un «8». Nodo de inicio: UNKNOWN; nodos objetivo: dígitos 0–9 y NULL; camino final en verde.

Cómo se asigna el peso. La similitud sale del coste J: s = redondeo(10 − 9·mín(J, 2)/2), limitada a [1, 10]. Un coste 0 da similitud 10 y un coste ≥ 2 da similitud 1. La arista hacia NULL pesa simil(τ) = 9, que es el «coste de rendirse»: un dígito solo gana si su similitud es mayor que la de NULL.

Pesos de las aristas y justificación (rasgos compartidos)

Imagen de entrada: H = 2, S = 1.00, E = 0, C = 1, densidad total = 0.224. Los prototipos son el promedio de las diez variantes de cada dígito (solo para esta tabla; el programa compara contra cada variante).

Arista	H proto	S proto	E proto	J	Peso s	c = 11−s	Rasgos compartidos (justificación)
UNKNOWN → 0	1.1	0.97	0.2	0.106	**10**	1	simetría, densidad, extremos, componentes

Arista	H proto	S proto	E proto	J	Peso s	c = 11−s	Rasgos compartidos (justificación)
UNKNOWN → 1	0.0	0.77	2.8	0.515	**8**	3	densidad, componentes
UNKNOWN → 2	0.0	0.65	2.2	0.544	**8**	3	densidad, componentes
UNKNOWN → 3	0.0	0.74	2.9	0.450	**8**	3	densidad, componentes
UNKNOWN → 4	1.0	0.67	1.5	0.559	**7**	4	densidad, componentes
UNKNOWN → 5	0.0	0.76	2.0	0.394	**8**	3	densidad, componentes
UNKNOWN → 6	1.0	0.84	1.0	0.233	**9**	2	densidad, componentes
UNKNOWN → 7	0.0	0.58	2.0	0.676	**7**	4	densidad, componentes
UNKNOWN → 8	2.0	0.98	0.0	0.090	**10**	1	huecos, simetría, densidad, extremos, componentes
UNKNOWN → 9	1.0	0.83	1.0	0.251	**9**	2	densidad, componentes
UNKNOWN → NULL	—	—	—	0.300	**9**	2	Umbral de rechazo τ

Lectura: el «8» se parece al 8 y al 0 (peso 10) porque comparten dos huecos o un contorno cerrado, simetría alta y ningún extremo; al 6 y al 9 (peso 9) los acerca un lazo cerrado y una densidad parecida en el centro; al 1, 2, 3, 4, 5 y 7 (pesos 7–8) solo los une la densidad y el número de componentes (todos tienen un trazo continuo).

Rasgos que justifican los pesos

Rasgo	Qué mide	Dígitos donde es más útil
Huecos (H)	Regiones de fondo rodeadas por el trazo	0, 6, 9 (≈ 1 hueco), 8 (2 huecos), 4 (≈ 1); sin hueco: 1, 2, 3, 5, 7
Simetría (S)	Parecido izquierda–derecha	Alta en 0 y 8 (≈ 0,97), baja en 7 y 2 (≈ 0,5–0,65)
Densidad regional (D)	Trazo por cada una de 16 regiones	Distingue formas con la misma topología (por ejemplo 6, 9 y 0)
Extremos (E)	Puntos terminales del esqueleto	0 y 8: ≈ 0; 6 y 9: ≈ 1; 1, 3 y 7: más de 2
Componentes (C)	Partes conexas del trazo	Casi siempre 1; sirve para detectar ruido o cortes

Los pesos reales dependen de cada imagen: una imagen distinta cambia la tabla. Estos rasgos no identifican por sí solos un dígito; su combinación ponderada es lo que forma el coste.

SECCIÓN 3. Algoritmos de búsqueda y aprendizaje

3.1 (a) Búsqueda no informada: BFS y DFS

Recorren el grafo de dígitos **sin información adicional**: no miran qué candidato se parece más a la imagen antes de visitarlo.

BFS (primero en anchura). Usa una **cola**. Se mete UNKNOWN; se extrae y se encolan sus 11 sucesores en el orden 0, 1, …, 9, NULL; se visitan en ese orden y, en cada visita, se calcula el coste real J del candidato.

Orden de visita: UNKNOWN → 0 → 1 → 2 → 3 → 4 → 5 → 6 → 7 → 8 → 9 → NULL

DFS (primero en profundidad). Usa una **pila**. Se apilan los sucesores en el mismo orden y se extrae el último, de modo que el recorrido sale invertido. Como todos los sucesores son hojas, la «profundidad» no llega a profundizar más allá de un nivel.

Orden de visita: UNKNOWN → NULL → 9 → 8 → 7 → 6 → 5 → 4 → 3 → 2 → 1 → 0

Cuando termina el recorrido se elige el candidato de menor coste (y NULL si ninguno cumple $J \leq \tau$).

Característica	BFS	DFS
Estructura	Cola (FIFO)	Pila (LIFO)
Usa heurística	No	No
Nodos verificados aquí	11 de 11	11 de 11
Complejidad	O(V + E) = O(11 + 11)	O(V + E) = O(11 + 11)
Garantiza menor coste	No por sí mismo (solo menor nº de aristas)	No
Depende del orden de los vecinos	Sí	Sí

Ventajas: sencillos, lineales y completos. Desventajas: visitan candidatos poco prometedores y su orden no tiene relación con la imagen.

3.2 (b) Búsqueda heurística: A*

A* ordena la frontera por **f(n) = g(n) + h(n)**, donde g(n) es el coste acumulado desde UNKNOWN y h(n) una estimación del coste que falta hasta el objetivo. Para que h(n) tenga sentido, el recorrido **UNKNOWN → d → objetivo** se modela en dos tramos:

- **g(d)** = coste de los rasgos de densidad (regionales y total): ya están medidos al expandir UNKNOWN.
- **r(d)** = coste de verificar el candidato: huecos, simetría, extremos y componentes.
- El coste del camino completo es g(d) + r(d) = **J(I, d)**. La arista UNKNOWN → NULL cuesta τ.

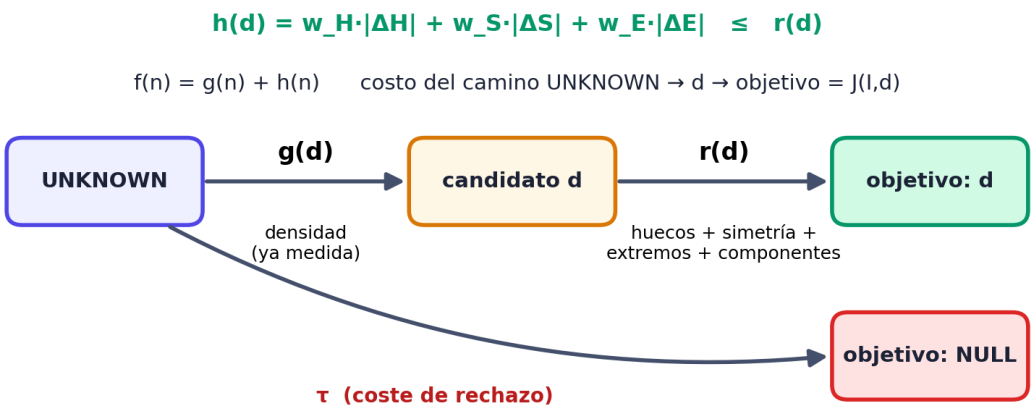


Figura 3. Modelo de A*: dos tramos por candidato y NULL como objetivo con coste τ.

Definición de h(n) basada en rasgos (huecos, simetría y endpoints):

$h(d) = w_H \cdot |\tilde{H}_I - \tilde{H}_d| + w_S \cdot |S_I - S_d| + w_E \cdot |\tilde{E}_I - \tilde{E}_d|$ (h = 0 en UNKNOWN, NULL y en los objetivos)

Admisibilidad. Una heurística es admisible si $0 \le h(n) \le h^*(n)$, siendo $h^*(n)$ el coste real mínimo restante. Desde un candidato d solo hay un camino al objetivo, así que $h^*(d) = r(d) = J(I,d) - g(d)$. Como r(d) contiene exactamente los términos de h más el término de componentes $w_C \cdot |\tilde{C}_I - \tilde{C}_d| \ge 0$ (los pesos son no negativos), se cumple $h(d) \le r(d) = h^*(d)$. Con diez variantes por dígito el razonamiento se mantiene: $J - g \ge \text{mín}_v\, r_v \ge \text{mín}_v\, h_v = h(d)$. Por tanto h es **admisible**.

Consistencia. Para la arista d → objetivo: $h(d) \le r(d) + h(\text{objetivo}) = r(d)$. Para UNKNOWN → d: $h(\text{UNKNOWN}) = 0 \le g(d) + h(d)$. Al ser admisible y consistente, A* con la prueba «extraer un objetivo de la frontera» devuelve el camino de menor coste, es decir, el dígito de menor J, o NULL si todos superan τ.

Traza real de A* con la imagen del «8»

La frontera inicial contiene los diez dígitos con f = g + h y NULL con f = τ = 0,30. En cada paso se extrae el menor f; si es un candidato, se verifica (su f pasa a ser el coste real J) y vuelve a la frontera.

Paso	Nodo extraído	f	Acción
1	0	0.056	Verifica: coste real J = 0.106
2	8	0.081	Verifica: coste real J = 0.090
3	8	0.090	Es objetivo: A* termina

A* solo verificó 2 de los 11 sucesores; los demás quedaron en la frontera con f mayor que el coste del objetivo hallado. BFS y DFS verifican los 11.

Valores f = g + h de todos los candidatos para esa imagen:

Dígito	g	h	f = g + h	J real
0	0.056	0.000	0.056	0.106
8	0.081	0.000	0.081	0.090
6	0.098	0.121	0.219	0.233
9	0.121	0.125	0.246	0.251
5	0.173	0.221	0.394	0.394
3	0.185	0.240	0.425	0.450
1	0.238	0.199	0.436	0.515

Dígito	g	h	f = g + h	J real
4	0.318	0.219	0.536	0.559
2	0.281	0.257	0.538	0.544
7	0.342	0.305	0.647	0.676

Se cumple f ≤ J en todas las filas, como exige la admisibilidad.

3.3 (c) Algoritmos genéticos

El AG ajusta la importancia relativa de los rasgos, es decir, los pesos de la función de coste. No aprende nuevos rasgos.

- **Cromosoma.** Vector de cinco pesos X = [w_H, w_S, w_D, w_E, w_C], cada uno en [0, 3] (por ejemplo X = [1,2; 1,0; 2,0; 0,8; 1,0]).
- **Población inicial.** 18 individuos: uno con los pesos manuales de partida y el resto aleatorios.
- **Fitness.** Porcentaje de aciertos sobre un conjunto de imágenes: Fitness(X) = aciertos(X) / total · 100. Se entrena con 100 imágenes y se valida con otras 60 distintas, para detectar sobreajuste.
- **Selección.** Torneo de tamaño 3 y elitismo (los 2 mejores pasan sin cambios).
- **Cruce.** Uniforme: cada peso del hijo se toma de uno de los dos padres con probabilidad 0,5. Ejemplo: A = [1,0; 1,2; 0,8; 1,1; 0,9] y B = [1,3; 0,9; 1,1; 0,7; 1,2] pueden producir H = [1,0; 0,9; 0,8; 0,7; 1,2].
- **Mutación.** Con probabilidad 0,2 por peso se suma un ruido aproximadamente gaussiano (±0,8 como máximo) y se acota a [0, 3].
- **Criterio de parada.** Se detiene al llegar a 12 generaciones, al alcanzar 100 % de fitness o si el mejor no mejora durante 6 generaciones.

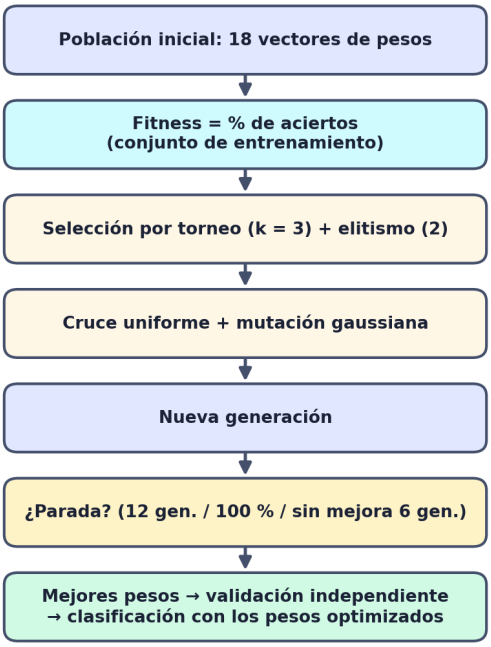


Figura 4. Ciclo del algoritmo genético.

Ejecución real del programa (botón «Optimizar pesos (AG)», semilla fija 7)

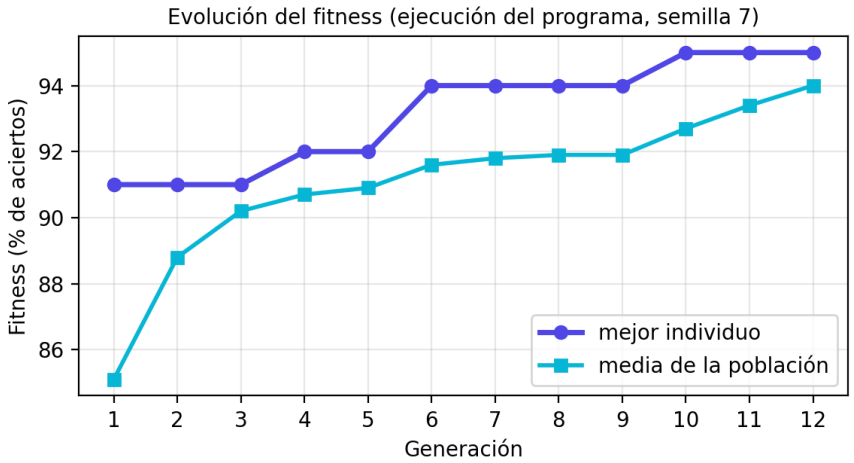


Figura 5. Fitness por generación en la ejecución del programa.

Resultado de esa ejecución: pesos iniciales [1,2; 1; 2; 0,8; 1] con 89,0 % en entrenamiento y **81,7 % en validación**; mejor cromosoma [0,13; 0,10; 2,48; 0,05; 1,63] con 95,0 % en entrenamiento y **90,0 % en validación**. Los pesos se aplican solo si no empeoran la validación.

Ventajas: no requiere derivadas y permite explorar combinaciones de pesos. Limitaciones: no garantiza el óptimo global, depende de que el conjunto de evaluación sea representativo y puede dar resultados distintos entre ejecuciones (aquí se fija la semilla para que sea reproducible).

3.4 (d) Flujo completo: rasgos + búsqueda + aprendizaje

El sistema une tres componentes: la **extracción de rasgos** (convierte la imagen en un vector), la **búsqueda** en el grafo (A*, BFS o DFS comparan ese vector con los prototipos usando el coste J) y el **aprendizaje** (el AG ajusta los pesos w de J).

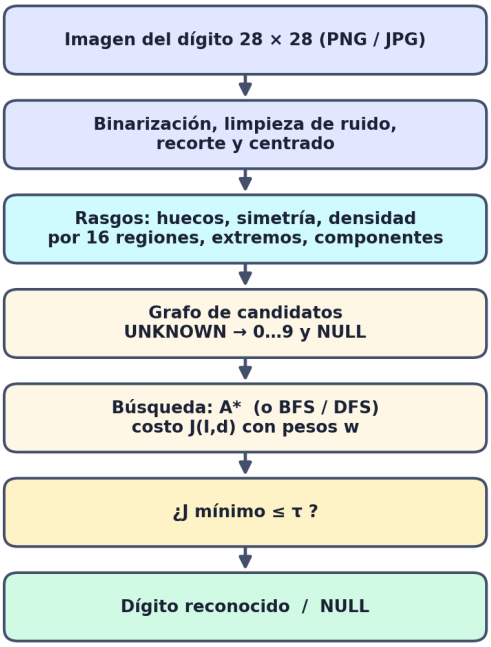


Figura 6. Flujo de clasificación de una imagen.

Cómo el AG mejora la clasificación. Con pesos manuales algunos rasgos pesan demasiado (por ejemplo, el número de huecos confunde 6, 9 y 0). El AG prueba combinaciones y se queda con las que aciertan más: sube el peso de la densidad regional ($w_D \approx 4$) y de los componentes, y baja el de huecos y extremos, que son rasgos más ruidosos. Para medir la mejora se comparan ambos conjuntos de pesos sobre las mismas imágenes de prueba:

Evaluación	Pesos manuales	Pesos del AG
500 imágenes nuevas (semilla 321), sin umbral	79,8 %	**96,0 %**
25 pruebas del programa (semilla 2026), sin umbral	21 / 25 (84 %)	**24 / 25 (96 %)**
Ejecución del AG integrado (validación, semilla 7)	81,7 %	**90,0 %**

Los pesos [0,13; 0,56; 4,21; 0,12; 2,39] provienen de una ejecución más larga del mismo AG (población 40, 50 generaciones, 600 imágenes de entrenamiento); las 25 pruebas y las 500 imágenes de evaluación usan semillas distintas a las de entrenamiento.

SECCIÓN 4. Programa funcional en HTML

El programa es una página **HTML + JavaScript** de un solo archivo (*index.html*), sin instalar nada: se abre con doble clic o se publica en GitHub Pages. Cumple los cinco requisitos:

Requisito	Dónde está en el programa
1. Cargar una imagen 28 × 28	Botón de archivo (PNG/JPG) y ejemplos; muestra la imagen original y la binaria normalizada.
2. Extraer rasgos (huecos, simetría, densidad por regiones, endpoints)	Panel «Rasgos extraídos»: H, S, densidad 4 × 4, densidad total, E y C.
3. Clasificar 0–9 + NULL con el grafo de búsqueda	Panel «Búsqueda y clasificación»: A*, BFS o DFS, umbral τ ajustable.
4. Animación del recorrido (nodo actual, visitados, camino final)	Panel del grafo de 12 nodos: nodo actual resaltado, visitados en azul, camino final en verde, registro paso a paso.
5. Generar y evaluar 25 pruebas (0–9 repetidos, con variaciones de sitio)	Botón «Evaluar 25 pruebas»: tabla con miniatura, esperado, predicho y variación.

Además incluye la optimización de pesos con el AG y la comparación entre pesos manuales y optimizados.



Figura 7. Encabezado de la aplicación.

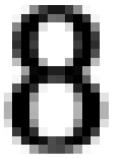
1 Imagen de entrada (28×28)

Choose File No file chosen

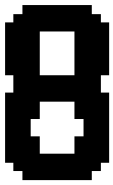
Usar imagen de ejemplo

Dígito 8

Original 28×28



Normalizada y binaria



PNG o JPG. Se detecta el fondo, se separa el trazo, se recorta y se centra en 28×28.

2 Rasgos extraídos

Huecos (H)	2
Simetría izq.-der. (S)	1.000
Densidad por regiones (D) 4×4	0.00 0.31 0.31 0.00 0.00 0.61 0.61 0.00 0.00 0.57 0.57 0.00 0.00 0.31 0.31 0.00
Densidad total	0.224
Extremos / endpoints (E)	0
Componentes conexos (C)	1

Figura 8. Carga de imagen (izquierda) y rasgos extraídos (derecha).

3 Búsqueda y clasificación

Algoritmo A* **Umbral τ** 0.30

Velocidad Lenta

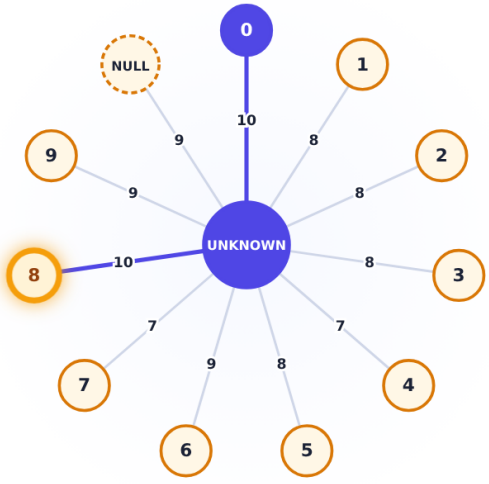
Clasificar y animar

Dígito 8

Mejor dígito: 8 (J = 0.090) · nodos verificados: 2/11 · J ≤ τ (0.3): se acepta.

```
Frontera inicial: f(d) = g(d) + h(d) para
cada dígito; NULL entra con f = τ.
Visita 0: g=0.056 h=0.000 → J=0.106
(f=0.056 → real 0.106)
Visita 8: g=0.081 h=0.000 → J=0.090
(f=0.081 → real 0.090)
Se extrae el objetivo 8 con f = 0.090 (el
menor de la frontera): A* termina.
A* expandió 2 de 11 vecinos gracias a
h(n).
Resultado: 8   Camino: UNKNOWN → 8
```

4 Animación del grafo de búsqueda



Nodo actual: 8 · J = 0.090

generado (frontera) nodo actual visitado resultado camino final

Peso de cada arista = similitud de rasgos (1–10). Coste de la arista = 11 − similitud.

Figura 9. Clasificación con A* (izquierda: resultado y registro) y animación en curso (derecha: nodo actual en naranja, visitado en azul).

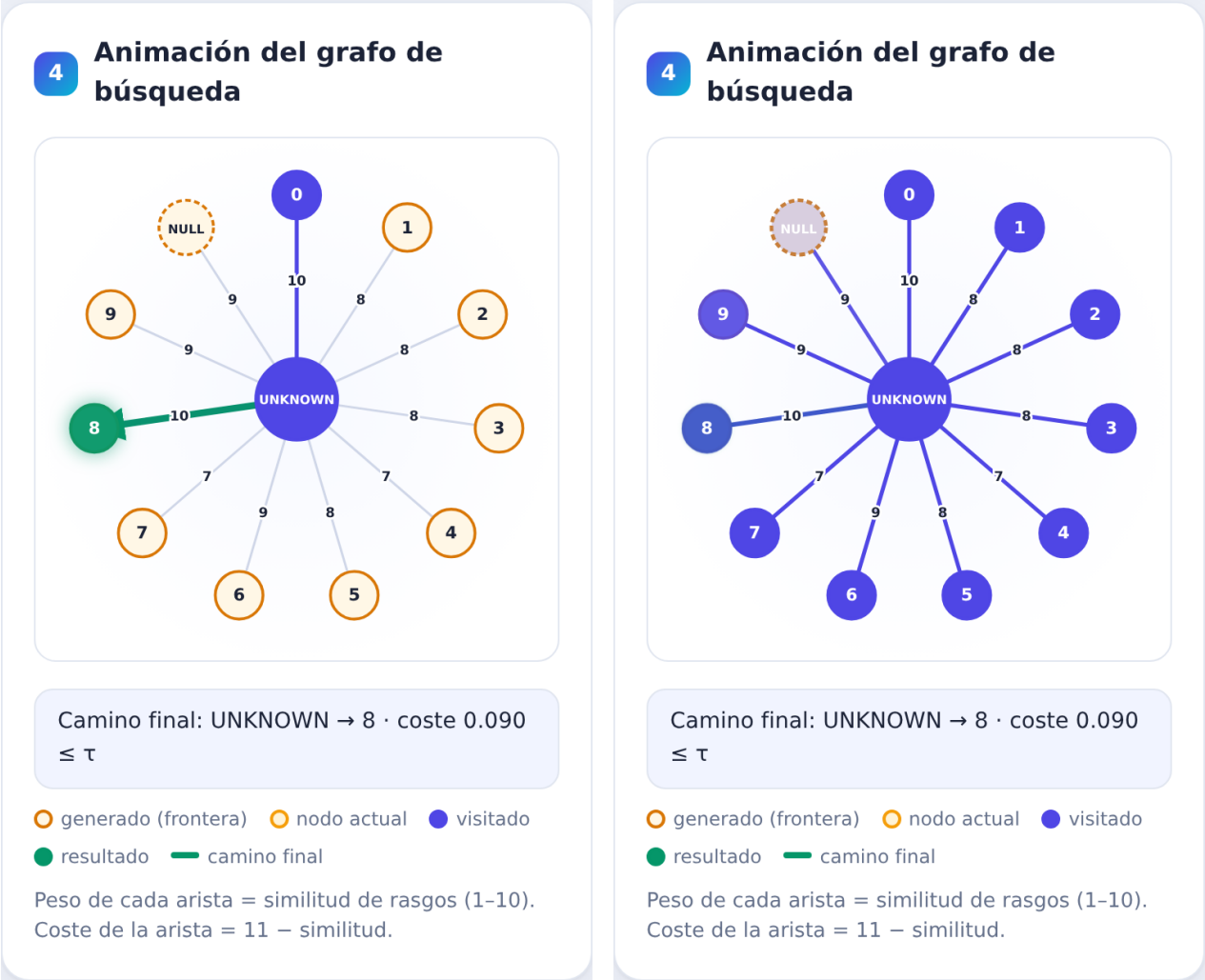


Figura 10. Camino final con A* (izquierda, 2 nodos verificados) y con BFS (derecha, los 11 nodos verificados).

5 Costes, pesos de aristas y rasgos compartidos

DÍGITO	G	H	F = G+H	J REAL	SIMILITUD (1–10)	COSTE 11−S	RASGOS COMPARTIDOS
8	0.081	0.000	0.081	0.090	10	1	huecos, simetría, densidad, extremos, componentes
0	0.056	0.000	0.056	0.106	10	1	simetría, densidad, extremos, componentes
6	0.098	0.121	0.219	0.233	9	2	densidad, componentes
9	0.121	0.125	0.246	0.251	9	2	densidad, componentes
5	0.173	0.221	0.394	0.394	8	3	densidad, componentes
3	0.185	0.240	0.425	0.450	8	3	densidad, componentes
1	0.238	0.199	0.436	0.515	8	3	densidad, componentes
2	0.281	0.257	0.538	0.544	8	3	densidad, componentes
4	0.318	0.219	0.536	0.559	7	4	densidad, componentes
7	0.342	0.305	0.647	0.676	7	4	densidad, componentes
NULL	—	—	0.300	0.300	9	2	umbral de rechazo τ

g = coste de los rasgos de densidad · h = estimación con huecos, simetría y extremos · f = g + h · J = coste real (g + h + componentes).

Figura 11. Tabla de costes g, h, f, J, pesos de arista (similitud 1–10, coste 11 − s) y rasgos compartidos.

Caso NULL

Una imagen que no es un dígito (la letra M o ruido aleatorio) tiene un coste mínimo muy superior a τ = 0,30 (0,91 para la letra M y ≈ 0,84 para el ruido del ejemplo), así que A* extrae NULL de la frontera sin verificar ningún dígito.



Figura 12. Resultado NULL para la letra M.

Evaluación con 25 imágenes de prueba

El botón genera siempre las mismas 25 imágenes (semilla fija): los dígitos 0–9 repetidos 2,5 veces, cada uno con un **desplazamiento aleatorio de posición** (±4 píxeles) y una segunda variación que rota entre ninguna, grosor, ruido, tipografía y tamaño. Resultado con los pesos del AG y τ = 0,30:

24 / 25 aciertos	**96 %** precisión	**0** resultados NULL

#	Imagen	Esperado	Predicho	J	Resultado	Variación
1		0	0	0.00	**Acierto**	posición (+0, -2)
2		1	1	0.17	**Acierto**	posición (+1, +1) + grosor fino +0.5
3		2	2	0.18	**Acierto**	posición (+1, +2) + ruido 1.6 %
4		3	3	0.00	**Acierto**	posición (-1, +1) + fuente Georgia
5		4	4	0.20	**Acierto**	posición (-4, -1) + tamaño 20 px
6		5	5	0.00	**Acierto**	posición (-3, -2)
7		6	6	0.21	**Acierto**	posición (+0, +2) + grosor negrita +1
8		7	7	0.03	**Acierto**	posición (-3, +1) + ruido 2.0 %
9		8	8	0.00	**Acierto**	posición (+2, +2) + fuente Arial
10		9	9	0.14	**Acierto**	posición (-1, -4) + tamaño 26 px
11		0	0	0.00	**Acierto**	posición (+3, +4)
12		1	1	0.26	**Acierto**	posición (-1, +2) + grosor negrita +1
13		2	2	0.18	**Acierto**	posición (+4, -1) + ruido 2.6 %
14		3	3	0.01	**Acierto**	posición (+0, -4) + fuente Arial
15		4	4	0.11	**Acierto**	posición (-3, +1) + tamaño 26 px
16		5	5	0.00	**Acierto**	posición (-1, +2)
17		6	6	0.21	**Acierto**	posición (-3, -2) + grosor negrita +1

#	Imagen	Esperado	Predicho	J	Resultado	Variación
18		7	7	0.01	**Acierto**	posición (-2, +1) + ruido 1.0 %
19		8	8	0.00	**Acierto**	posición (-4, +0) + fuente Georgia
20		9	0	0.19	**Fallo**	posición (+0, -1) + tamaño 22 px
21		0	0	0.00	**Acierto**	posición (-3, +1)
22		1	1	0.19	**Acierto**	posición (-4, -4) + grosor negrita +0.5
23		2	2	0.15	**Acierto**	posición (+3, +2) + ruido 1.1 %
24		3	3	0.00	**Acierto**	posición (-1, +4) + fuente Courier New
25		4	4	0.16	**Acierto**	posición (-4, +1) + tamaño 22 px

El fallo (prueba 20: esperado 9, predicho 0) es una confusión entre dos dígitos con un lazo cerrado (9 y 0). Son imágenes sintéticas generadas por el propio programa: miden la coherencia del sistema, no su precisión con escritura manuscrita real.

Ejecución y publicación en GitHub Pages

Ejecutar localmente: descomprimir el ZIP y abrir *index.html* con el navegador. Pasos sugeridos: «Usar imagen de ejemplo» → «Clasificar y animar» → «Evaluar 25 pruebas» → «Optimizar pesos (AG)».

Publicar:

- Crear un repositorio público en GitHub (por ejemplo *reconocedor-digitos*).
- Subir *index.html* y *README.md* a la raíz del repositorio.
- En **Settings → Pages**, elegir la rama *main* y la carpeta */ (root)*, y guardar.
- GitHub mostrará la URL pública, con la forma https://TU-USUARIO.github.io/reconocedor-digitos/ (la URL real la genera GitHub para tu cuenta).

Limitaciones

- Los prototipos se generan con fuentes del navegador (cinco tipografías, normal y negrita); las 25 pruebas y las 500 imágenes de evaluación usan esas mismas familias de fuentes con cambios de posición, grosor, ruido y tamaño. Con otras tipografías o con escritura manuscrita real la precisión sería menor y no se midió.
- Algunas letras parecidas a dígitos (por ejemplo S, Z, I, X) tienen un coste menor que τ y se aceptan como dígito; M, W y el ruido aleatorio sí dan NULL.
- El umbral $\tau = 0{,}30$ se calibró con estas imágenes sintéticas: el coste máximo de una clasificación correcta en las 500 imágenes fue 0,29.
- El programa está hecho en JavaScript para funcionar en GitHub Pages sin instalar nada.
- Para un reconocedor robusto haría falta evaluar con un conjunto independiente de dígitos manuscritos, como MNIST.

Conclusión

El reconocimiento de dígitos se plantea como una búsqueda en un grafo de 12 nodos donde la imagen desconocida se compara con diez candidatos y un nodo de rechazo. BFS y DFS recorren todos los candidatos sin usar información; A*, con una heurística admisible basada en huecos, simetría y extremos, llega al mismo resultado óptimo verificando muy pocos candidatos (2 de 11 en el ejemplo). El algoritmo genético ajusta los pesos de los rasgos y eleva la precisión medida de 79,8 % a 96,0 % sobre 500 imágenes nuevas. La aplicación entregada permite ver cada paso: rasgos, grafo animado, tabla de costes, 25 pruebas y la optimización.